D. Bad Luck Island

time limit per test: 2 seconds
memory limit per test: 256 megabytes

The Bad Luck Island is inhabited by three kinds of species: r rocks, s scissors and p papers. At some moments of time two random individuals meet (all pairs of individuals can meet equiprobably), and if they belong to different species, then one individual kills the other one: a rock kills scissors, scissors kill paper, and paper kills a rock. Your task is to determine for each species what is the probability that this species will be the only one to inhabit this island after a long enough period of time.

Input

The single line contains three integers r, s and p ($1 \le r, s, p \le 100$) — the original number of individuals in the species of rock, scissors and paper, respectively.

Output

Print three space-separated real numbers: the probabilities, at which the rocks, the scissors and the paper will be the only surviving species, respectively. The answer will be considered correct if the relative or absolute error of each number doesn't exceed 10^{-9}.

Examples

input	Copy
2 2 2	

output	Copy
0.333333333333 0.333333333333 0.333333333333	

input	Copy
2 1 2	

output	Copy
0.150000000000 0.300000000000 0.550000000000	

input	Copy
1 1 3	

output	Copy
0.057142857143 0.657142857143 0.285714285714	

We have three species:

- Rocks (R)
- Scissors (S)
- Papers (P)

Rules (rock-paper-scissors):

- Rock kills Scissors
- Scissors kill Paper
- Paper kills Rock

At each step:

- Two random individuals meet
- All unordered pairs are equally likely
- If they are different species, → one kills the other
- Population decreases by exactly 1

This process continues until only one species remains.

We want three probabilities:

- Probability that rocks are the only survivors
- Probability that scissors are the only survivors
- Probability that papers are the only survivors

Why this is NOT a simulation problem

- The process is random
- Number of steps is random
- Outcomes branch exponentially

Simulation would:

- Be slow
- Be inaccurate
- Fail precision constraints

So we must use probability DP.

Core modeling idea

At any moment, the system is completely described by:

$$(r, s, p)$$

That is:

- number of rocks
- number of scissors
- number of papers

This is a state.

Total number of possible meetings

If total individuals $r + s + p$,
number of unordered pairs is:

$$\binom{r+s+p}{2}$$

But we only care about cross-species meetings, because:

Same species → nothing happens.

Possible killing events from a state (r, s, p)

There are exactly three types of killing events:

- Rock meets Scissors
  - Number of such pairs = $r * s$
  - Rock kills Scissors
  - New state: $(r, s-1, p)$
- Scissors meets Paper
  - Number of such pairs = $s * p$
  - Scissors kills Paper
  - New state: $(r, s, p-1)$
- Paper meets Rock
  - Number of such pairs = $p * r$
  - Paper kills Rock
  - New state: $(r-1, s, p)$

Let:

$$T = r * s + s * p + p * r$$

be the total number of effective (killing) pairs.

Transition probabilities

Given we are in state (r, s, p):

- Probability scissors die:
$$\frac{r * s}{T}$$
- Probability paper dies:
$$\frac{s * p}{T}$$
- Probability rock dies:
$$\frac{p * r}{T}$$

These are conditional probabilities, conditioned on a killing happening.

Define the DP states

Let:

$dp[r][s][p]$ = probability of ever reaching state (r, s, p)

We start from:

$dp[R][S][P] = 1$

We want to propagate probabilities downward until only one species remains.

DP recurrence

For every state (r, s, p) with $r+s+p \ge 1$:

$$dp[r][s][p] \to \begin{cases} dp[r][s-1][p] \;+=\; dp[r][s][p] \cdot \frac{r*s}{T}, \\ dp[r][s][p-1] \;+=\; dp[r][s][p] \cdot \frac{s*p}{T}, \\ dp[r-1][s][p] \;+=\; dp[r][s][p] \cdot \frac{p*r}{T}. \end{cases}$$

Code

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int R, S, P;
    cin >> R >> S >> P;

    static double dp[101][101][101] = {};
    dp[R][S][P] = 1.0;

    for (int r = R; r >= 0; r--) {
        for (int s = S; s >= 0; s--) {
            for (int p = P; p >= 0; p--) {
                if (r == 0 && s == 0) continue;
                if (s == 0 && p == 0) continue;
                if (p == 0 && r == 0) continue;

                double cur = dp[r][s][p];
                if (cur == 0.0) continue;

                double T = (double)r*s + (double)s*p + (double)p*r;
                if (T == 0) continue;

                if (r > 0 && s > 0)
                    dp[r][s-1][p] += cur * (r * s) / T;
                if (s > 0 && p > 0)
                    dp[r][s][p-1] += cur * (s * p) / T;
                if (p > 0 && r > 0)
                    dp[r-1][s][p] += cur * (p * r) / T;
            }
        }
    }

    double pr = 0, ps = 0, pp = 0;
    for (int i = 1; i <= R; i++) pr += dp[i][0][0];
    for (int i = 1; i <= S; i++) ps += dp[0][i][0];
    for (int i = 1; i <= P; i++) pp += dp[0][0][i];

    cout << fixed << setprecision(12)
         << pr << " " << ps << " " << pp << "\n";
}
```